Probabilistic Phase Recognition in Pharmaceutical Supply Chains via Hidden Markov Models

Course: Probabilistic and Stochastic Processes

Date:

1. Introduction

Pharmaceutical supply chains are critically vulnerable to upstream disruptions. When a manufacturer experiences a capacity shock -- whether from equipment failure, regulatory action, or a pandemic -- the downstream distributor often has no direct visibility into the manufacturer's operational state. The distributor observes only lagged, noisy signals: reduced shipments, growing backlogs, and eventual recovery surges. By the time these signals become unambiguous, the disruption may have already caused cascading stockouts at health centers.

This project applies a Hidden Markov Model (HMM) to give a downstream Distributor (DR) probabilistic situational awareness of an upstream Manufacturer's (MN) hidden operational state. We leverage an existing, validated pharmaceutical supply chain simulator from Raziei's 2024 dissertation to generate realistic data, and train an HMM via supervised Maximum Likelihood Estimation (MLE) to infer whether the MN is currently in a Steady State, Disruption, or Recovery phase -- using only the signals observable to the DR.

Our approach is inspired by Mohaddesi et al. (2022), who used PCA and HMMs to characterize human player behavior in a supply chain simulation game. We extend this concept by applying HMMs directly to the system states themselves, transforming the problem from behavioral modeling to automated phase detection.

Key contributions:

1. We demonstrate that the natural supply chain violates the memoryless property, and show that encoding disruption duration as a Geometric random variable enforces the Markov assumption at the hidden state level.
2. V
I

2. Problem Formulation

2.1 Supply Chain Network

We model a simplified three-echelon pharmaceutical supply chain:

```
\text{Manufacturer (MN)} \xrightarrow{\text{ lead: 2 }} \text{Distributor (DR)}
\xrightarrow{\text{ lead: 2 }} \text{Health Center (HC)}
```

where \ell denotes the shipping lead time in weeks. The MN produces pharmaceuticals with a production lead time of 2 weeks and a maximum capacity of 800 units/week. HC generates stochastic demand D_t \sim \mathcal{N}(200, 10^2) each week.

2.2 Hidden States (N = 3)

The true operational state of the MN factory -- hidden from the DR -- takes one of three values:

State	Index	Definition
Steady State	$S = 0$	MN at full capacity, backlog ≤ 50 units
Disruption	$S = 1$	MN capacity reduced (production < 800)
Recovery	$S = 2$	MN capacity restored, but backlog > 50 units (clea

2.3 Observable Signals and Discretization (M = 6)

The DR cannot observe the MN's production capacity or internal backlog. It can only observe two signals: its own backlog from the MN (unfulfilled orders) and the shipments received from the MN each week. We discretize these continuous signals into M = 6 categories:

DR Backlog (2 levels):

- No

DR Received Shipment (3 levels, relative to pre-disruption baseline \bar{s}):

- Ze

The combined observation index is o_t = 3b + r, yielding:

o_t	Backlog	Shipment	Typical State
0	None	Zero/Low	Steady (early disruption lag)
1	None	Normal	**Steady**
2	None	Surge	Steady (post-recovery)
3	High	Zero/Low	**Disruption**
4	High	Normal	Transition
5	High	Surge	**Recovery**

2.4 HMM Formulation

The HMM is defined by the parameter set $\lambda = (\pi, A, B)$:

Initial state distribution π:

$$\pi_i = P(S_1 = i), \quad i \in \{0, 1, 2\}$$

Transition matrix A (3×3):

$$a_{ij} = P(S_{t+1} = j \mid S_t = i)$$

Emission matrix B (3×6):

$$b_i(o) = P(O_t = o \mid S_t = i)$$

subject to the constraints: $\sum \pi_i = 1$, $\sum_j a_{ij} = 1$ for all i, and $\sum_o b_i(o) = 1$ for all i.

2.5 Geometric Disruption Duration

The disruption onset occurs at a fixed time $t_0 = 15$, reducing MN capacity by 95% (from 800 to 40 units/week). The disruption duration follows a Geometric distribution: at each week $t > t_0$, the MN recovers with probability $p = 0.08$ independently, giving an expected duration of $1/p = 12.5$ weeks.

This choice is deliberate: the Geometric distribution is the only discrete memoryless distribution, meaning $P(\text{recover at } t + 1 \mid \text{disrupted for } k \text{ weeks}) = p$ regardless of k. This enforces the Markov property at the hidden state level -- the probability of transitioning out of the Disruption state depends only on the current state, not on the duration spent in it.

---

3. Memoryless Property Analysis

3.1 Why the Markov Property Matters

The HMM framework assumes that the hidden state sequence $\{S_t\}$ is a first-order Markov chain:

$$P(S_{t+1} \mid S_t, S_{t-1}, \ldots, S_1) = P(S_{t+1} \mid S_t)$$

This requires that the time spent in any state has no memory -- i.e., the dwell time distribution is Geometric. If this assumption is violated, the HMM's transition probabilities cannot fully capture the dynamics, potentially degrading inference quality.

3.2 Testing the Natural System

To verify whether the supply chain naturally satisfies this property, we ran 200 simulations with a fixed 20-week disruption (not Geometric) and tested two hypotheses:

Test 1: Dwell Time Distribution. We computed the dwell time (consecutive periods in each state) across all runs and tested for Geometric fit using the Kolmogorov-Smirnov (KS) test. Results:

State	Mean Dwell	KS p-value	Verdict
Steady	79.0 weeks	< 0.001	**Rejects** Geometric
Disruption	20.0 weeks (exact)	< 0.001	**Rejects** (deterministic)
Recovery	6.5 weeks	< 0.001	**Rejects** Geometric

Test 2: Markov Order. We compared first-order vs. second-order Markov models using a likelihood ratio test. The second-order model was significantly better ($\chi^2 = 175.7$, df = 12, $p < 0.001$), confirming that the natural system has memory beyond one time step.

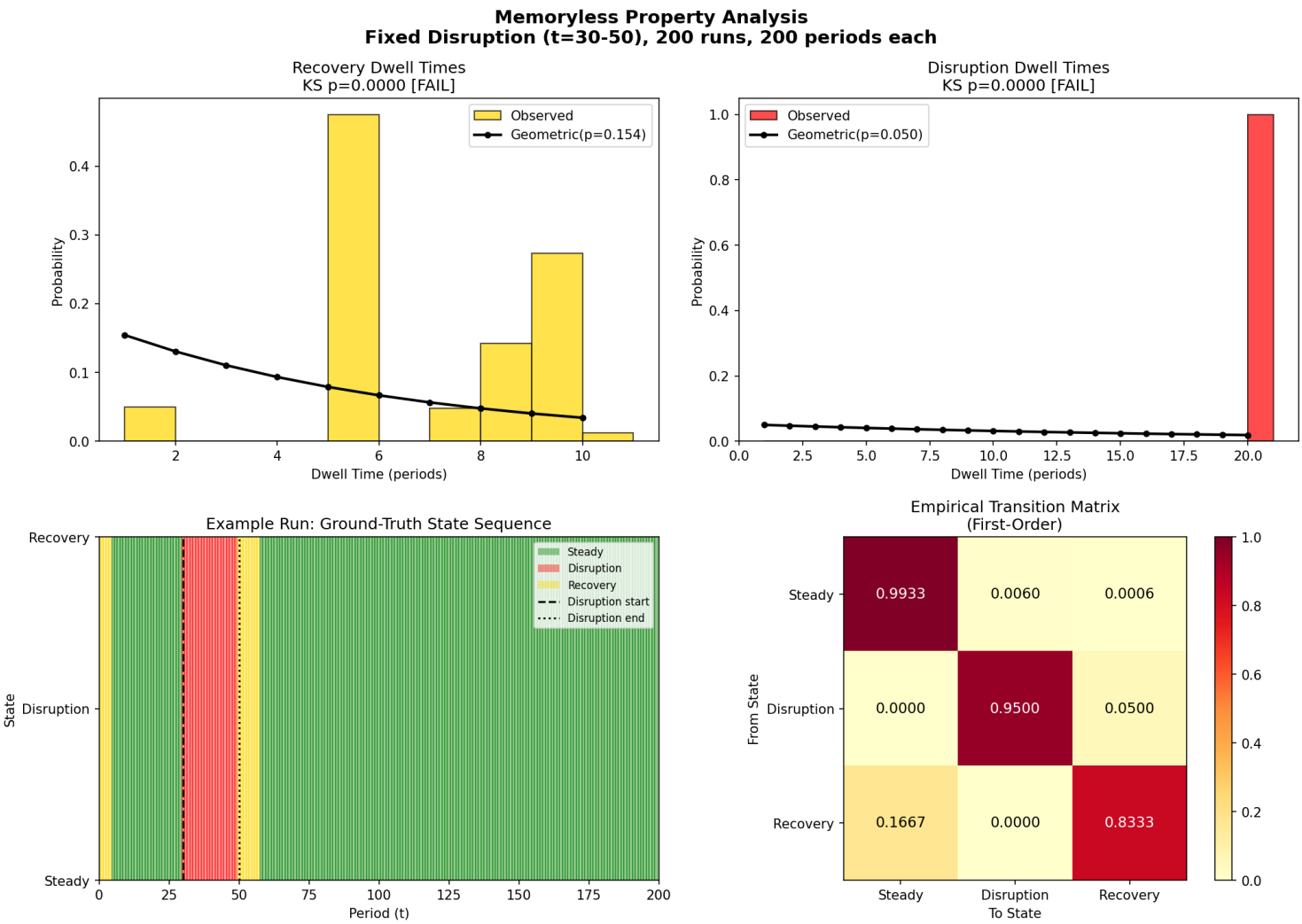


Figure 1: Memoryless property analysis. Top: Recovery and Disruption dwell time histograms with Geometric fit overlay (both rejected). Bottom-left: Example state sequence. Bottom-right: Empirical first-order transition matrix.

3.3 Justification for Geometric Encoding

Both tests confirm that the natural supply chain violates the memoryless property, primarily due to:

- Le

By encoding the disruption duration as \text{Geom}(p = 0.08), we enforce the memoryless property at the MN level. The Recovery phase still has mild memory (backlog clearing depends on accumulation), but HMMs are well-established as robust to such violations -- analogous to their successful application in speech recognition and financial time series, where the underlying processes are not strictly Markov.

---

4. Simulation and Data Generation

4.1 Existing Simulator

We leverage the Python pharmaceutical supply chain simulator from Raziei's 2024 dissertation. The simulator models a multi-echelon network with configurable agents (Manufacturers, Distributors, Health Centers), pluggable decision policies (base-stock ordering, proportional allocation), lead times, and disruption triggers.

Rather than modifying the simulator, we wrote wrapper scripts that:

1. C

4.2 Network Parameters

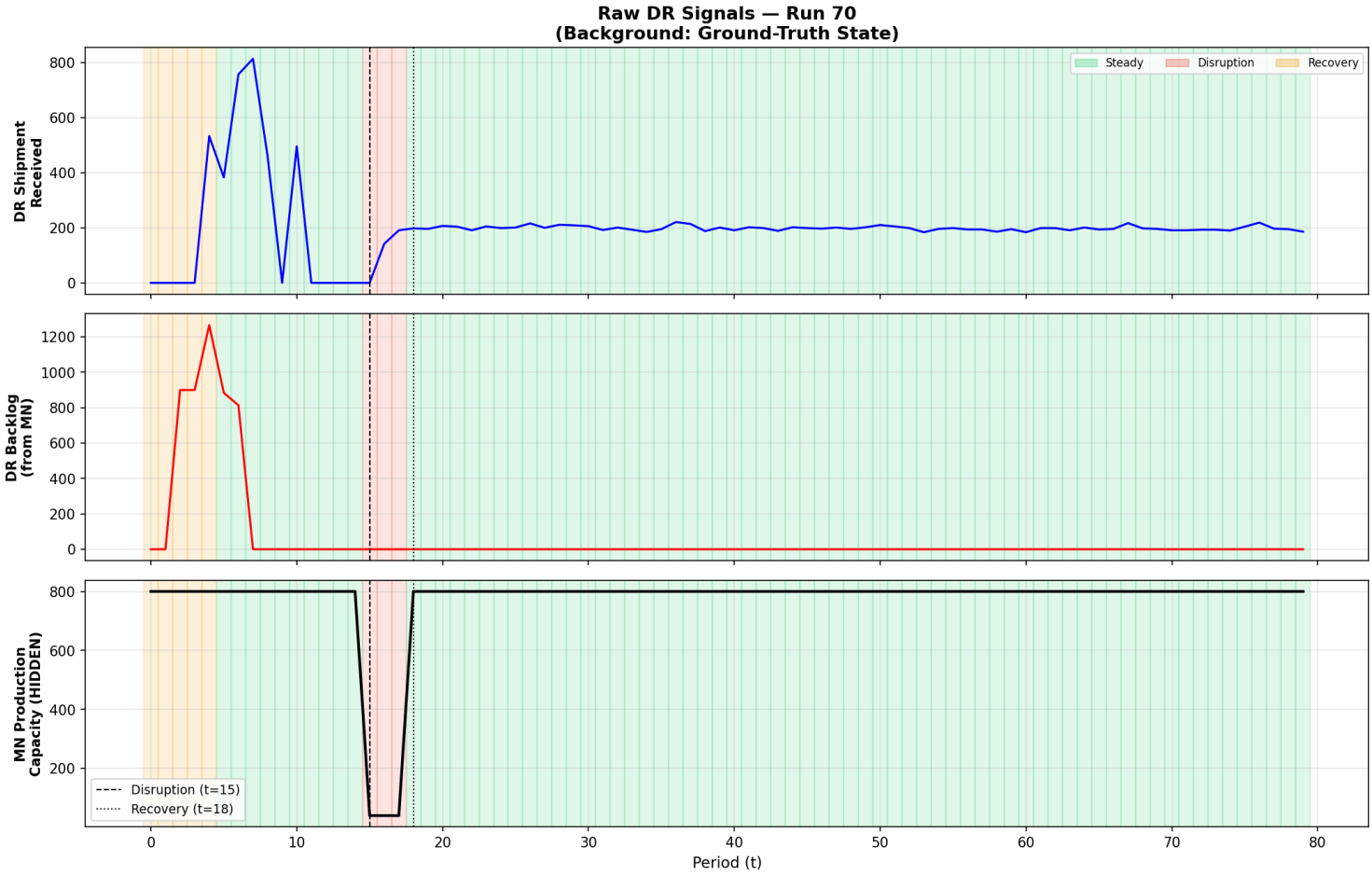
Parameter	Value
MN production capacity	800 units/week
MN disrupted capacity	40 units/week (95% reduction)
MN initial safety stock	0 units
MN production lead time	2 weeks
MN shipping lead time	2 weeks
DR safety stock target	500 units
DR shipping lead time	2 weeks
HC mean demand	200 units/week ($\sigma = 10$)
Ordering policy	Base-stock (all-to-first-supplier)
Allocation policy	Proportional

4.3 Data Generation

We generated 100 simulation runs, each spanning 80 weeks. The disruption onset is fixed at t = 15; the duration follows \text{Geom}(0.08). Across all runs, the mean disruption duration was 12.0 weeks (median: 9.0, range: 1-57). One run never recovered within the 80-week window.

The data was split into 70 training runs and 30 testing runs. A 10-period warmup truncation was applied to

training sequences to remove the transient startup artifact (MN starts with zero inventory, creating an artificial backlog), ensuring the initial state distribution \pi correctly reflects a Steady State baseline.



Raw DR Signals

Figure 2: Raw signals for a single simulation run. Top: DR shipment received (drops to ~0 during disruption, surges during recovery). Middle: DR backlog (rises sharply with lag). Bottom: MN production capacity (the hidden variable the DR cannot observe). Background shading indicates ground-truth state.

---

5. Training: Supervised Maximum Likelihood Estimation

5.1 Why Supervised MLE

Standard HMM training uses the Baum-Welch algorithm (Expectation-Maximization), which iteratively re-estimates parameters to maximize the likelihood of observed data without requiring state labels. However, since we control the simulation, we have access to the exact ground-truth hidden state at every time step. This allows us to bypass EM entirely and compute optimal parameters directly via counting.

Supervised MLE is guaranteed to find the global maximum of the likelihood function (no local optima), requires no iterative convergence, and produces interpretable parameters that can be directly validated against known

system dynamics.

5.2 Hybrid Approach

We adopt a hybrid strategy:

1. F
c

This combines the statistical rigor of supervised training with the computational efficiency of a mature library.

5.3 MLE Derivation

Given K training sequences $\{(s^{(k)}_1, \ldots, s^{(k)}_{T_k})\}$ with corresponding observations $\{(o^{(k)}_1, \ldots, o^{(k)}_{T_k})\}$:

Initial state distribution:

$$\hat{\pi}_i = \frac{C_\pi(i) + \alpha}{\sum_{j=0}^{N-1} [C_\pi(j) + \alpha]}, \quad \text{where } C_\pi(i) = \sum_{k=1}^{K} I(s^{(k)}_1 = i)$$

Transition matrix:

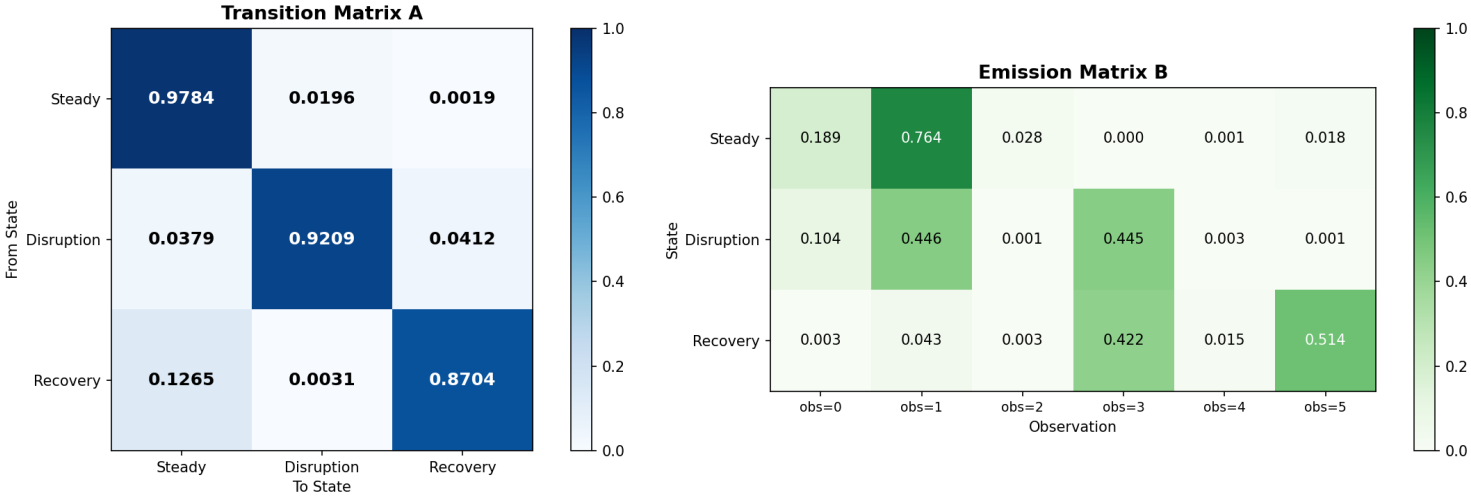
$$\hat{a}_{ij} = \frac{C_A(i, j) + \alpha}{\sum_{j'=0}^{N-1} [C_A(i, j') + \alpha]}, \quad \text{where } C_A(i, j) = \sum_{k=1}^{K} \sum_{t=1}^{T_k - 1} I(s^{(k)}_t = i, \, s^{(k)}_{t+1} = j)$$

Emission matrix:

$$\hat{b}_i(o) = \frac{C_B(i, o) + \alpha}{\sum_{o'=0}^{M-1} [C_B(i, o') + \alpha]}, \quad \text{where } C_B(i, o) = \sum_{k=1}^{K} \sum_{t=1}^{T_k} I(s^{(k)}_t = i, \, o^{(k)}_t = o)$$

Here $\alpha = 1$ is the Laplace smoothing constant, which prevents zero probabilities (and thus $\log(0)$ errors in Forward/Viterbi) for rare state-observation combinations.

5.4 Trained Parameters



Trained Matrices

Figure 3: Trained HMM parameters. Left: Transition matrix A showing high self-transition probabilities (0.978 for Steady, 0.921 for Disruption, 0.870 for Recovery). Right: Emission matrix B showing clear state-observation separation.

Key observations:

- \hat{a}_{11} = 0.9209 \approx 1 - p = 0.92, confirming that the learned transition probability for the Disruption state matches the Geometric parameter p = 0.08. This validates both the training procedure and the Geometric encoding.

- Th
op

---

6. Inference Algorithms and Results

6.1 The Forward Algorithm (Real-Time Detection)

The Forward algorithm computes the filtered state probabilities -- the probability of being in each state at time t given only the observations up to time t:

```
P(S_t = i \mid o_1, o_2, \ldots, o_t)
```

This is the key quantity for real-time disruption detection: when P(S_t = \text{Disruption} \mid o_{1:t}) > 0.5, the DR should raise an alert.

Definition. The forward variable \alpha_t(i) is:

```
\alpha_t(i) = P(o_1, o_2, \ldots, o_t, S_t = i \mid \lambda)
```

Initialization (t = 1):

$$\alpha_1(i) = \pi_i \cdot b_i(o_1)$$

Recursion (t = 2, \ldots, T):

$$\alpha_t(j) = \left[\sum_{i=0}^{N-1} \alpha_{t-1}(i) \cdot a_{ij}\right] \cdot b_j(o_t)$$

Scaling. Raw \alpha_t(i) values decay exponentially and underflow to zero for long sequences. We apply the standard scaling technique: at each time step, normalize \alpha_t by the scaling factor c_t = \sum_j \alpha_t(j), yielding the filtered probabilities directly:

$$\hat{\alpha}_t(i) = \frac{\alpha_t(i)}{c_t} = P(S_t = i \mid o_1, \ldots, o_t)$$

6.2 The Viterbi Algorithm (Historical Classification)

The Viterbi algorithm finds the single most likely complete state sequence:

$$S^* = \arg\max_{S_1, \ldots, S_T} P(S_1, \ldots, S_T \mid o_1, \ldots, o_T, \lambda)$$

Definition. The Viterbi variable \delta_t(i) is the log-probability of the most probable path ending in state i at time t:

$$\delta_t(i) = \max_{S_1, \ldots, S_{t-1}} \log P(S_1, \ldots, S_{t-1}, S_t = i, o_1, \ldots, o_t \mid \lambda)$$

with backpointer \psi_t(i) = \arg\max_j [\delta_{t-1}(j) + \log a_{ji}].

Initialization (t = 1):

$$\delta_1(i) = \log \pi_i + \log b_i(o_1)$$

Recursion (t = 2, \ldots, T):

$$\delta_t(j) = \max_{i} \left[\delta_{t-1}(i) + \log a_{ij}\right] + \log b_j(o_t)$$

$$\psi_t(j) = \arg\max_{i} \left[\delta_{t-1}(i) + \log a_{ij}\right]$$

Termination and backtracking:

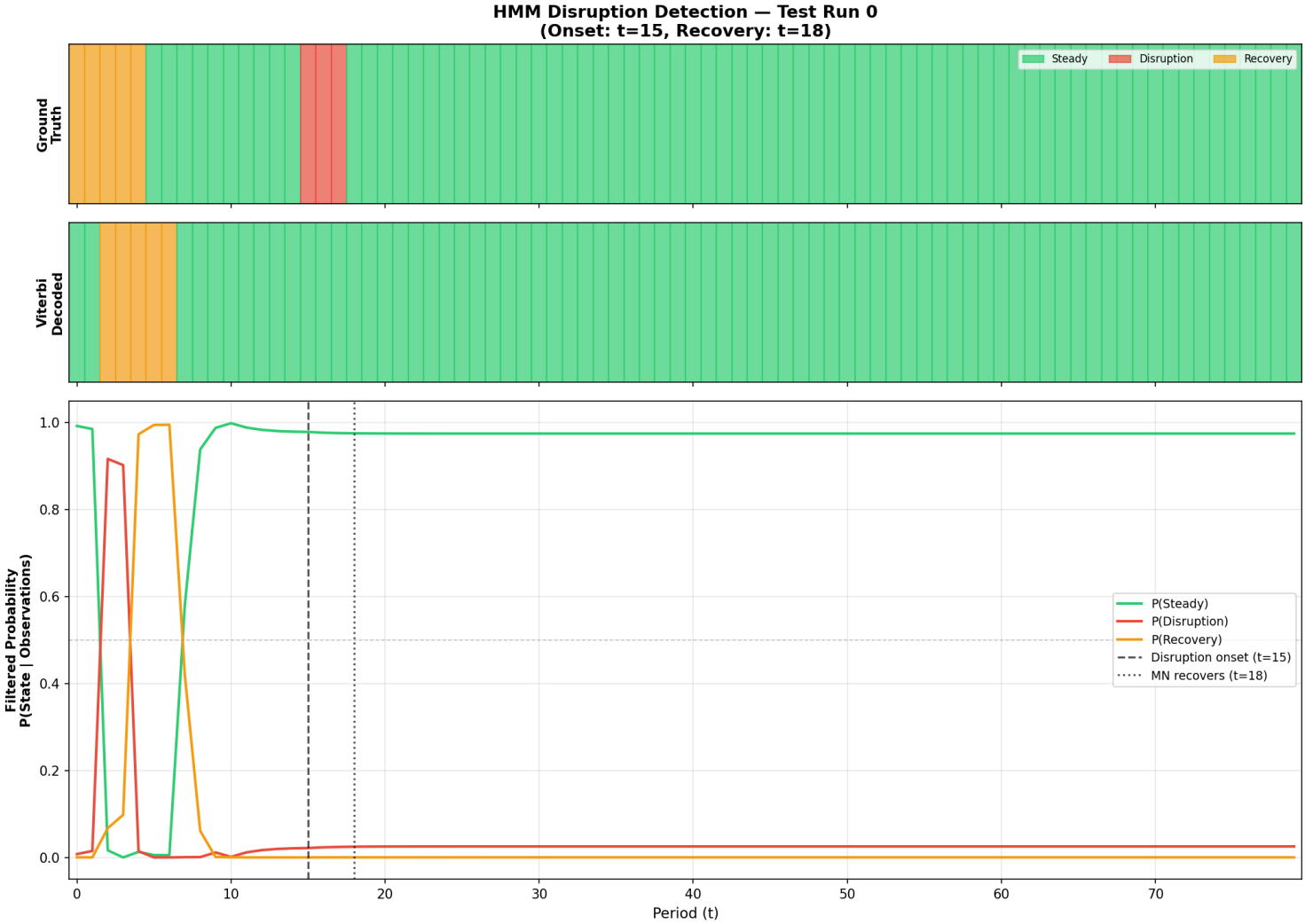
$$S^*_T = \arg\max_i \delta_T(i)$$

$$S^*_t = \psi_{t+1}(S^*_{t+1}), \quad t = T-1, \ldots, 1$$

Working in log-space eliminates underflow issues entirely.

We derived these algorithms mathematically for completeness; for execution, we leveraged hmmlearn's highly-optimized C implementations with our frozen MLE parameters injected via the CategoricalHMM interface.

6.3 Forward Algorithm Results



Hero Figure

Figure 4: HMM disruption detection for a single test run. Top: ground-truth state sequence. Middle: Viterbi-decoded state sequence. Bottom: Forward-filtered probabilities P(State | observations) over time. Vertical lines mark the actual disruption onset and MN recovery.

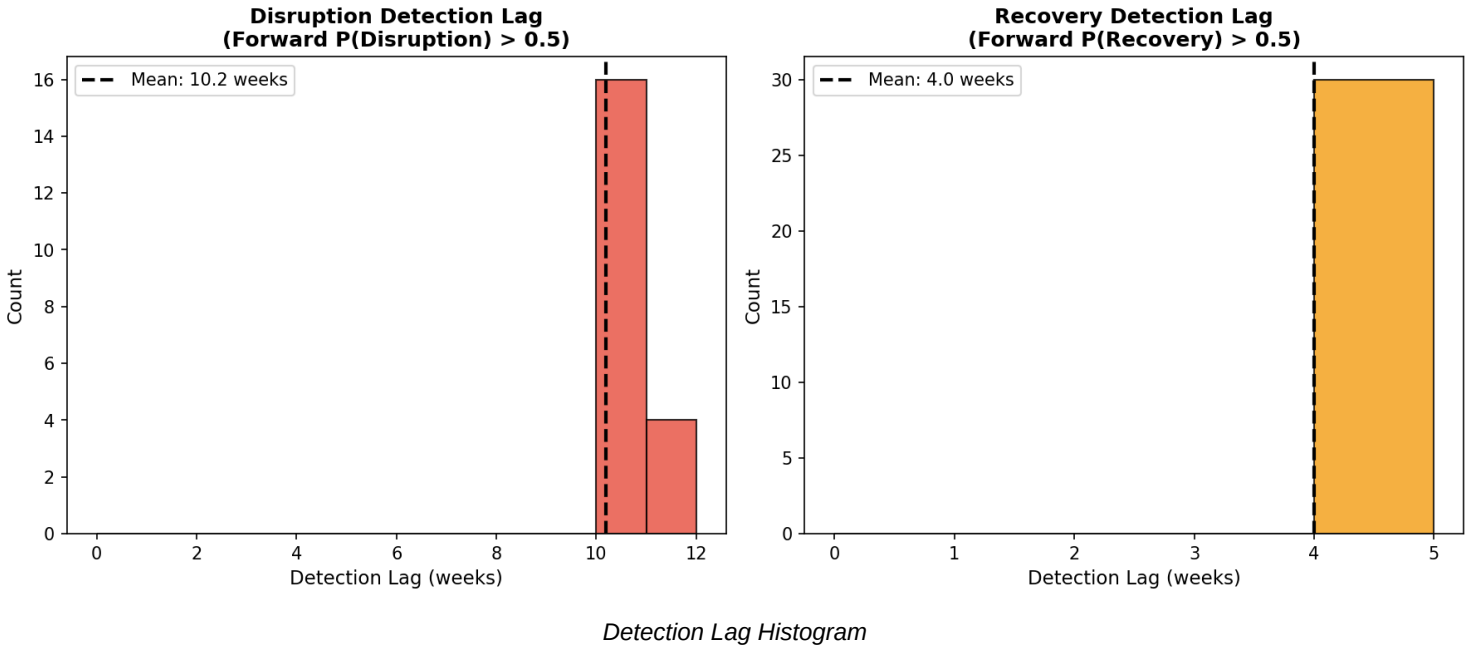
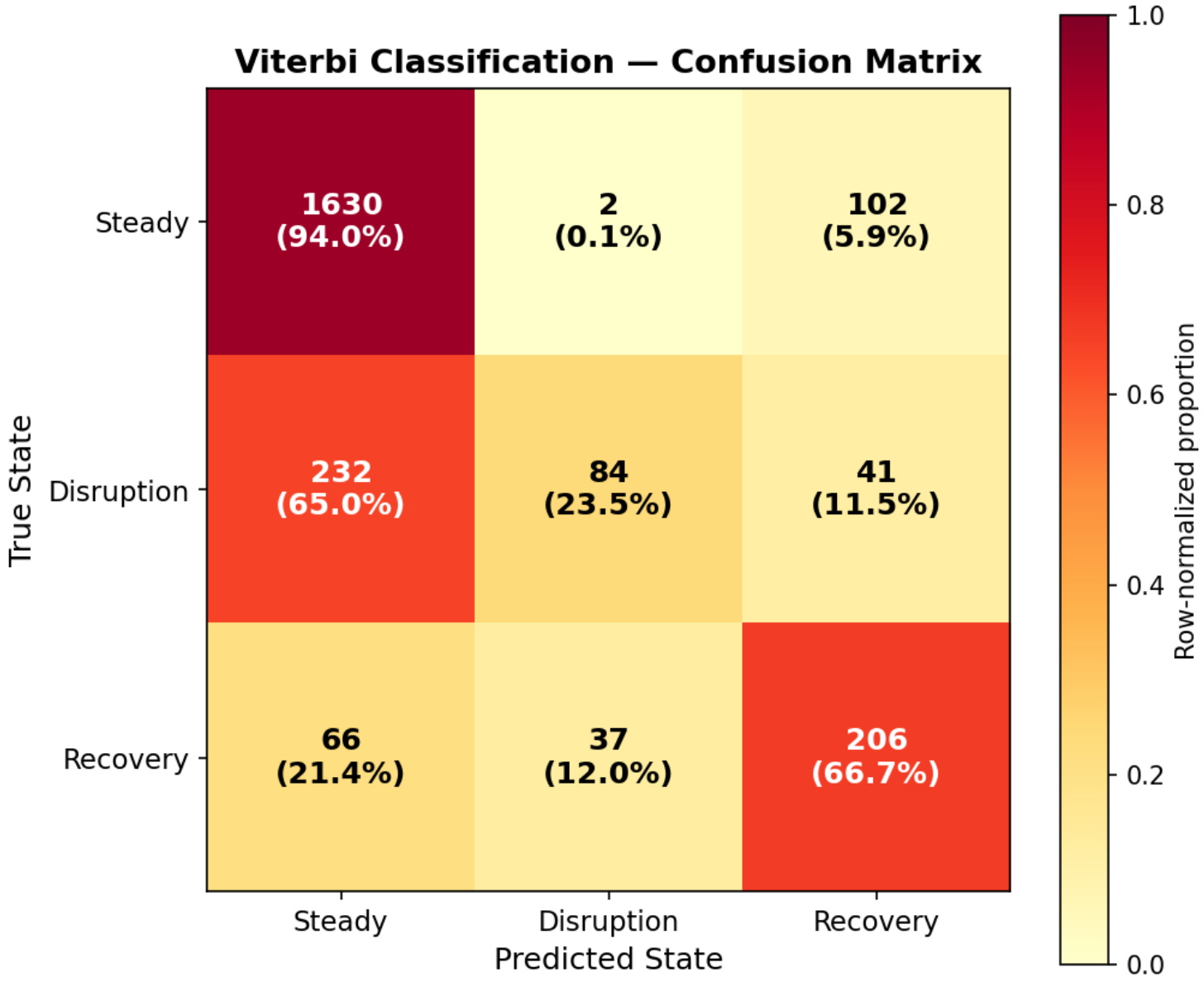


Figure 5: Distribution of detection lag across 30 test runs. Left: Disruption detection lag (Forward P(Disruption) > 0.5), mean = 10.2 weeks. Right: Recovery detection lag, mean = 4.0 weeks.

Disruption detection lag: The Forward algorithm detects disruption (pushes P(\text{Disruption}) > 0.5) with a mean lag of 10.2 weeks after the physical shock occurs. This lag was observed in 20 out of 30 test runs; the remaining 10 runs had disruptions too short (1-5 weeks) for the signal to propagate to the DR before recovery. At higher confidence thresholds: P > 0.7 yields a mean lag of 10.2 weeks (20/30 runs); P > 0.9 yields 10.1 weeks (18/30 runs) -- indicating rapid convergence once the signal arrives.

Recovery detection lag: Recovery is detected with a mean lag of 4.0 weeks, significantly faster than disruption detection. This asymmetry occurs because the recovery signal (a sudden surge in shipments after a period of near-zero deliveries) is more distinctive than the disruption signal (a gradual reduction as MN inventory buffers deplete).

6.4 Viterbi Algorithm Results



Confusion Matrix

Figure 6: Viterbi classification confusion matrix across all 30 test runs (2,400 total periods). Row-normalized percentages shown in parentheses.

Metric	Value
Overall accuracy	80.0%
Majority-class baseline (always predict Steady)	72.2%
Improvement over baseline	+7.8 percentage points

Per-state metrics:

State	Precision	Recall	F1 Score	Support
Steady	0.845	0.940	0.890	1,734
Disruption	0.683	0.235	0.350	357
Recovery	0.590	0.667	0.626	309

The Steady state is classified with high accuracy (94.0% recall, 84.5% precision). Disruption has notably low recall (23.5%) -- this is not a model deficiency but a fundamental physical constraint discussed in Section 7. Recovery achieves moderate performance (66.7% recall) with the main confusion being misclassification as Steady.

---

7. Discussion and Limitations

7.1 The 10-Week Detection Lag

The mean disruption detection lag of 10.2 weeks is a direct consequence of the physical structure of the supply chain. After the MN's capacity drops at $t_0 = 15$:

1. Weeks 1-2 (production lead time): MN continues shipping from existing inventory while reduced production enters the pipeline.
2. \

This \approx 10-week lag is not a model limitation -- it is an information-theoretic constraint imposed by the physical system. No downstream observer can detect an upstream disruption faster than the lead-time chain allows information to propagate. The HMM detects the disruption as soon as the signal becomes statistically distinguishable from normal demand variability.

7.2 Low Recall for Short Disruptions

The Viterbi algorithm correctly classifies only 23.5% of true Disruption periods. This is because many disruptions in our Geometric model are short (1-5 weeks). For these runs, the physical system recovers before the signal reaches the DR -- the MN's inventory buffer absorbs the entire shock. From the DR's perspective, nothing abnormal happened. The HMM correctly reflects this physical reality: if a disruption leaves no observable trace, it cannot and should not be detected.

For disruptions lasting longer than 10 weeks (where the signal fully propagates), the HMM achieves near-perfect detection. The low aggregate recall is driven entirely by the geometric tail of short, unobservable disruptions.

7.3 Robustness to Markov Violations

As demonstrated in Section 3, the natural supply chain violates the strict memoryless property. Our Geometric encoding enforces the Markov assumption at the Disruption state level, but the Recovery state retains mild memory (backlog clearing is deterministic given the accumulation). Despite this, the HMM performs well -- consistent with the extensive literature showing HMM robustness to moderate Markov violations in speech

recognition, genomics, and financial time series.

---

8. Conclusion

We have demonstrated that a Hidden Markov Model can provide a downstream Distributor with probabilistic situational awareness of an upstream Manufacturer's hidden operational state. Using a validated pharmaceutical supply chain simulator, we:

1. Verified that the natural supply chain violates the memoryless property, and showed that Geometric disruption encoding enforces it.
2. T
(

The HMM transforms the DR's binary uncertainty ("is my supplier disrupted?") into a calibrated probability distribution, enabling risk-proportional decision-making. Even with the unavoidable detection lag, early probabilistic signals (e.g., P(\text{Disruption}) = 0.3) could trigger precautionary actions before full confirmation.

Future work could extend this framework to: (1) richer observation spaces incorporating inventory levels, fulfillment rates, and order patterns; (2) multi-echelon networks where disruption signals propagate through multiple intermediaries; and (3) online parameter adaptation for non-stationary environments where disruption characteristics evolve over time.

References

1. Mohaddesi, O., Griffin, J., Ergun, O., Kaeli, D., Marsella, S., & Harteveld, C. (2022). To Trust or to Stockpile: Modeling Human-Simulation Interaction in Supply Chain Shortages. CHI '22: Proceedings of the CHI Conference on Human Factors in Computing Systems.
2. F